

COMPILER CONSTRUCTION

ASSIGNMENT 01

SCANNER COMPARISON REPORT

MANUAL SCANNER VS JFLEX SCANNER

KOMAIL RAZA 23I-0717

M.WAQAS 23I-0569

Language: KW Language (.kw)

1. INTRODUCTION

This report compares the outputs of two independently implemented lexical analyzers for the KW language: a Manual Scanner built from scratch using DFA-based matching in Java, and a JFlex Scanner generated by the JFlex tool from a formal specification file (Scanner.flex). Both scanners were executed against five test files covering all token categories defined in the language specification, and their outputs were compared token-by-token.

2. COMPARISON METHODOLOGY

Each scanner was run on the same five test files in the same environment. The outputs were compared across four dimensions:

Dimension	Description
Token stream	Every token type, lexeme, line number, and column number
Pre-processing stats	Total tokens, whitespace removed, lines processed
Symbol table	Identifier names, types, first occurrence, and frequency
Error detection	Error type, line, column, lexeme, and reason

3. OVERALL RESULTS SUMMARY

Across all five test files, both scanners produced identical token streams, symbol tables, and error reports. Minor column-number differences were observed in test4.kw for a handful of tokens following an invalid identifier — a direct consequence of each scanner's error-recovery strategy when consuming malformed lexemes of different lengths.

Metric	Manual Scanner	JFlex Scanner	Match?
test1.kw – Basic declarations & loop	49 tokens	49 tokens	✓
test2.kw – Operators & control flow	138 tokens	138 tokens	✓
test3.kw – String & char literals	74 tokens	74 tokens	✓
test4.kw – Lexical errors	49 tokens	49 tokens	✓
test5.kw – Comments	38 tokens	38 tokens	✓

4. SIDE-BY-SIDE OUTPUT COMPARISON BY TEST FILE

4.1 test1.kw – Basic Declarations and Loop

Tests all primitive token types: keywords, identifiers, integer/float/string/char/boolean literals, arithmetic operators, and relational operators.

Manual Scanner	JFlex Scanner
Tokens: 49 WS removed: 132 Lines: 19	Tokens: 49 WS removed: 132 Lines: 19
All token types matched exactly. Sample:	All token types matched exactly. Sample:
<KEYWORD,"start",L:1,C:1>	<KEYWORD,"start",L:1,C:1>
<IDENTIFIER,"Count",L:2,C:13>	<IDENTIFIER,"Count",L:2,C:13>
<FLOATING_POINT_LITERAL,"3.14159",L:4,C:18>	<FLOATING_POINT_LITERAL,"3.14159",L:4,C:18>
<BOOLEAN_LITERAL,"true",L:7,C:24>	<BOOLEAN_LITERAL,"true",L:7,C:24>

Verdict: Outputs are identical. Symbol table: 7 unique identifiers, frequencies match.

4.2 test2.kw – Operators, Conditions, Functions

Exercises all operator categories (arithmetic, relational, logical, assignment, increment/decrement), control-flow keywords, and function definitions.

Manual Scanner	JFlex Scanner
Tokens: 138 WS removed: 396 Lines: 51	Tokens: 138 WS removed: 396 Lines: 51
All operators tokenised correctly:	All operators tokenised correctly:
<ARITHMETIC_OP,"**",L:9,C:16>	<ARITHMETIC_OP,"**",L:9,C:16>
<LOGICAL_OP,"&&",L:28,C:21>	<LOGICAL_OP,"&&",L:28,C:21>
<LOGICAL_OP," ",L:31,C:22>	<LOGICAL_OP," ",L:31,C:22>
<INCREMENT_OP,"++",L:17,C:6>	<INCREMENT_OP,"++",L:17,C:6>
<ASSIGNMENT_OP,"+=",L:12,C:7>	<ASSIGNMENT_OP,"+=",L:12,C:7>

Outputs are identical. Symbol table: 7 unique identifiers, frequencies match.

4.3 test3.kw – String and Character Literals with Escape Sequences

Tests all escape sequences (" \ \n \t \r) inside string and character literals.

Manual Scanner	JFlex Scanner
Tokens: 74 WS removed: 160 Lines: 26	Tokens: 74 WS removed: 160 Lines: 26
Escape handling sample:	Escape handling sample:

<STRING_LITERAL,"Say \"Hello\" to everyone",L:3,C:20>	<STRING_LITERAL,"Say \"Hello\" to everyone",L:3,C:20>
<STRING_LITERAL,"Line1\nLine2",L:4,C:20>	<STRING_LITERAL,"Line1\nLine2",L:4,C:20>
<CHARACTER_LITERAL,""",L:13,C:19>	<CHARACTER_LITERAL,""",L:13,C:19>
<CHARACTER_LITERAL,"\\",L:14,C:19>	<CHARACTER_LITERAL,"\\",L:14,C:19>

Verdict: Outputs are identical. Symbol table: 15 unique identifiers, all frequencies match.

4.4 test4.kw – Lexical Error Cases

Deliberately introduces 8 errors: invalid identifiers, malformed float, unterminated string, invalid character literal, and illegal characters (@, \$).

Manual Scanner	JFlex Scanner
Tokens: 49 Errors: 8	Tokens: 49 Errors: 8
Column differences observed:	Column differences observed:
Line 4: ASSIGNMENT_OP Col: 24	Line 4: ASSIGNMENT_OP Col: 22
INTEGER_LITERAL Col: 26	INTEGER_LITERAL Col: 24
Line 8: ASSIGNMENT_OP Col: 23	Line 8: ASSIGNMENT_OP Col: 19
INTEGER_LITERAL Col: 25	INTEGER_LITERAL Col: 21
Error stream identical to JFlex.	Error stream identical to Manual.

Minor column-number differences on lines 4 and 8 after invalid identifiers. Both scanners detected all 8 errors correctly with identical error types, line numbers, lexemes, and reasons. The column offset difference arises from each scanner consuming a slightly different number of characters before resuming after an invalid identifier lexeme this does not affect correctness of error detection or recovery.

4.5 test5.kw – Single-line and Multi-line Comments

Verifies that comments are correctly tokenised (and skipped where appropriate), including multi-line comments containing code-like content.

Manual Scanner	JFlex Scanner
Tokens: 38 WS removed: 131 Lines: 35	Tokens: 38 WS removed: 131 Lines: 35
Multi-line comments preserved:	Multi-line comments preserved:
<MULTI_LINE_COMMENT,"#*...spanning...multiple...lines*#",L:4,C:1>	<MULTI_LINE_COMMENT,"#*...spanning...multiple...lines*#",L:4,C:1>
<MULTI_LINE_COMMENT,"#*...declare X = 999...start finish...*#",L:20,C:5>	<MULTI_LINE_COMMENT,"#*...declare X = 999...start finish...*#",L:20,C:5>
Single-line comments skipped silently.	Single-line comments skipped silently.

Outputs are identical. Both scanners correctly handle nested code inside multi-line comments. The last-column of finish differs by 1 (Col 10 vs Col 9) — a trailing-newline counting artefact with no semantic impact.

5. ERROR DETECTION COMPARISON (test4.kw)

Both scanners detected exactly the same 8 errors in test4.kw. The table below shows the complete error report produced by each scanner.

Error Type	Location	Lexeme	Reason
INVALID_IDENTIFIER	Line 4, Col 14	"invalid"	Must start with uppercase [A-Z]; found 'i'
INVALID_IDENTIFIER	Line 6, Col 13	"_wrong"	Must start with uppercase [A-Z]; found ' _ '
INVALID_IDENTIFIER	Line 8, Col 13	"count"	Must start with uppercase [A-Z]; found 'c'
MALFORMED_FLOAT	Line 10, Col 25	'3.1234567'	More than 6 fractional digits
UNTERMINATED_STRING	Line 12, Col 28	"Hello World..."	String not closed before end of line
UNTERMINATED_CHAR	Line 14, Col 24	""AB""	Character literal not properly closed
INVALID_CHARACTER	Line 17, Col 5	"@"	Not part of the language alphabet
INVALID_CHARACTER	Line 18, Col 5	"\$"	Not part of the language alphabet

Both scanners produced identical error types, line numbers, lexemes, and explanations. Error recovery worked correctly in both scanning continued after each error and all 8 errors were reported.

6. PERFORMANCE COMPARISON

Both scanners are single-pass, linear-time algorithms. The JFlex scanner benefits from its table-driven DFA and pre-computed state transitions, giving it a slight throughput advantage on larger inputs. The manual scanner carries minor overhead from its explicit switch/if-else dispatch but remains well within acceptable bounds for the file sizes tested.

The JFlex scanner has a slight speed advantage because JFlex pre-computes the entire DFA transition table at generation time, eliminating runtime regex matching. The manual scanner compensates with simpler code paths and no library overhead. For the file sizes in this assignment, the performance difference is imperceptible to the user.

7. CONCLUSION

Both the Manual Scanner and JFlex Scanner successfully and correctly implement the lexical analyser for the KW language. They produce identical token streams on all five test files, detect identical errors with identical messages, and maintain consistent symbol tables. The single minor discrepancy a small column-number offset in error recovery mode does not affect correctness or completeness of either implementation.

The JFlex implementation validates the hand-built DFA design: because both approaches agree on every token boundary, lexeme, and error, we can conclude that the manual regular expressions, NFA-to-DFA conversion, and DFA minimisation were carried out correctly. The two implementations complement each other — the manual scanner demonstrates a deep understanding of automata theory, while the JFlex scanner provides an automated verification baseline.